TP1 – Manipuler des données d'imagerie médicale

Apprentissage profond en imagerie médicale – Master de mathématiques

Informations générales

— **Langage** : Python (notebook Jupyter ou Google Colab).

— **Bibliothèques** : `numpy`, `matplotlib`, `pydicom`, `nibabel`, `SimpleITK`, `scikit-learn`, `monai` (facultatif pour certaines parties).

— **Livrable** : un notebook commenté, avec le code, les figures et les réponses aux questions (en texte, entre les cellules). Les questions marquées $\star$ demandent une réponse rédigée, pas seulement un résultat numérique.

Objectifs. Lire et inspecter des données DICOM et NIfTI, comprendre leur géométrie, les visualiser correctement, normaliser leurs intensités, les rééchantillonner, et mettre en place une séparation train/test *sans fuite d'information*. Ces étapes sont un préalable à tous les TP suivants.

Données.

— **DICOM** : fichiers d'exemple fournis par `pydicom` (par exemple `CT_small.dcm` et `MR_small.dcm`, accessibles avec la fonction `get_testdata_file` du module `pydicom.data`).

— **NIfTI** : jeux de données du *Medical Segmentation Decathlon* (volumes `.nii.gz` avec masques). Nous suggérons de prendre une tâche de petite taille (rate ou cur) ; `monai.apps.DecathlonDataset` permet de la télécharger. Si possible, comparer avec un second jeu CT (par exemple foie) pour la partie 5 (normalisation).

1 Mise en place

1. Installer et importer les bibliothèques. Vérifier la version de chacune.

```
import numpy as np
import matplotlib.pyplot as plt
import pydicom, nibabel as nib, SimpleITK as sitk
from pydicom.data import get_testdata_file
```

2 Lire et inspecter un fichier DICOM

1. Charger `CT_small.dcm` avec `pydicom.dcmread`. Afficher les métadonnées principales : modalité, taille de l'image, espacement entre pixels (`PixelSpacing`), épaisseur de coupe (`SliceThickness`) lorsqu'elle est présente, `RescaleSlope`, `RescaleIntercept`.
2. Récupérer `ds.pixel_array` et afficher son type, sa forme et ses valeurs minimale et maximale.
3. **[À faire]** Convertir les valeurs stockées en unités de Hounsfield : HU $=$ slope $\times$ valeur_stockée $+$ intercept. Vérifier que l'air est proche de -1000 et les tissus mous de l'ordre de quelques dizaines à une centaine d'unités.

4. Afficher l'image avec trois fenêtrages différents (par exemple poumon, tissus mous, os), en précisant centre et largeur de fenêtre : $[c - w/2,\ c + w/2]$.
   Pour les tissus tels aue les poumons les valeurs HU sont entre (-600, 1500), pour les tissus mous entre (40, 400) et pour les os entre (400, 1800).

⋆ Quelles informations d'identification potentielle contient l'en-tête DICOM ? Lister au moins cinq champs qu'il faudrait supprimer ou modifier avant de diffuser ce fichier.

```
ds = pydicom.dcmread(get_testdata_file("CT_small.dcm"))
img = ds.pixel_array.astype(np.float32)
slope = float(getattr(ds, "RescaleSlope", 1.0))
intercept = float(getattr(ds, "RescaleIntercept", 0.0))
# TODO: hu = ...
```

3 Lire un volume NIfTI : géométrie et orientation

On utilise ici la bibliothèque `nibabel` https://nipy.org/nibabel/tutorials.html. On pourra par exemple appliquer les questions à des images de rate disponibles dans le Medical Segmentation Challenge http://medicaldecathlon.com (Task09, Spleen).

Deux solutions pour lire ce fichier.
Solution 1 : on le telechqrge directement depuis le site du Medical Segmentation Challenge en utilisant des outils de `monai`.

```
from monai.apps import download_and_extract
import tempfile

# On verifie si un repertoire "MONAI_DATA_DIRECTORY" existe
# Si non, on cree un repertoire temporaire
directory = os.environ.get("MONAI_DATA_DIRECTORY")
if directory is not None:
    os.makedirs(directory, exist_ok=True)
root_dir = tempfile.mkdtemp() if directory is None else directory
print(root_dir)

# On telecharge et on decompresse les donnees
resource = "https://msd-for-monai.s3-us-west-2.amazonaws.com/Task09_Spleen.tar"
md5 = "410d4a301da4e5b2f6f86ec3ddba524e"

compressed_file = os.path.join(root_dir, "Task09_Spleen.tar")
data_dir = os.path.join(root_dir, "Task09_Spleen")
if not os.path.exists(data_dir):
    download_and_extract(resource, compressed_file, root_dir, md5, "md5")
```

Solution 2 : On télécharge le fichier compressé. On peut le décompresser sous python ou en extraire l'arborescence et les fichiers.

```
import tarfile
tar_path = "chemin/Task09_Spleen.tar" # adapter le chemin

# Définissez le dossier local où vous souhaitez enregistrer les 3 fichiers
output_dir = "chemin"
os.makedirs(output_dir, exist_ok=True)

# Compteur pour suivre le nombre d'extractions
nb_max = 500
file_count = 0
```

```
with tarfile.open(tar_path, 'r') as tar:
    for member in tar.getmembers():
        # On vérifie qu'il s'agit bien d'un fichier (on ignore les dossiers vides)
        if member.isfile():
            print(f"Extraction du fichier ({file_count + 1}/nb_max) : {member.name}")
            # Extraction physique sur le disque local
            tar.extract(member, path=output_dir)
            file_count += 1
            # Condition d'arrêt immédiat dès que le quota de nb_max fichiers est atteint
            if file_count == nb_max:
                print("Extraction terminée : les", nb_max," premiers fichiers ont été
    récupérés.")
                break
```

1. Charger un volume et son masque avec `nibabel`. Afficher la taille du tableau, le type de données, l'espacement des voxels (`header.get_zooms()`) et la matrice affine (`img.affine`) pour l'image https://nipy.org/nibabel/coordinate_systems.html.
   L'espacement est la distance physique mesurée entre les centres de deux voxels adjacents. La matrice affine permet de convertir instantanément les indices de la grille (i,j,k) en coordonnées physiques réelles (x,y,z) en millimètres (généralement dans le système RAS : Right, Anterior, Superior).

$$\begin{pmatrix} x_{physique} \\ y_{physique} \\ z_{physique} \\ 1 \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} & A_{13} & T_x \\ A_{21} & A_{22} & A_{23} & T_y \\ A_{31} & A_{32} & A_{33} & T_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} i \\ j \\ k \\ 1 \end{pmatrix}$$

2. Calculer l'orientation du volume avec `nib.aff2axcodes(img.affine)`. Que signifie par exemple `('R','A','S')` ou `('L','P','S')` ?
3. Vérifier que l'image et son masque ont la même forme, la même affine et le même espacement. Que se passerait-il sinon ?
4. Réorienter le volume dans une orientation canonique (`nib.as_closest_canonical`) et vérifier si l'affine a changé ou non.
   Dans quel(s) cas l'affine change t'elle ?
   Le contenu anatomique reste cohérent (superposer le masque sur l'image avant/après).
5. **[À faire]** Écrire une fonction `voxel_to_world(affine, ijk)` qui retourne les coordonnées en millimètres d'un voxel (i, j, k), puis calculer le volume du masque en millilitres : $V = N_{\text{voxels}} \times \prod_d s_d \ / \ 1000$.

⋆ Pourquoi le volume d'un organe ne peut-il pas être déduit du simple comptage de voxels ? Donner un exemple concret d'erreur si l'on ignore l'espacement.

4 Visualisation correcte

1. Créer une fonction utilitaire `show_slices(volume, spacing=None)` qui affiche les trois coupes centrales (axiale, coronale, sagittale) côte à côte.

```
def show_slices(vol, spacing=(1, 1, 1), cmap="gray", **kw):
    # vol: array (X, Y, Z)
    # spacing: voxel size along each axis
    # TODO: display the 3 central slices with correct aspect ratios
    pass
```

2. Afficher les trois coupes centrales (axiale, coronale, sagittale) d'un volume *sans* tenir compte de l'espacement, puis en le prenant en compte (paramètre `aspect` de `imshow`). Commenter la différence.
3. Superposer le masque de segmentation à l'image avec une transparence (`alpha`) et une carte de couleurs adaptées.
4. Choisir la coupe contenant le maximum de voxels segmentés plutôt que la coupe centrale. Que constatez-vous pour un organe de petite taille ?

⋆ Pourquoi est-il dangereux de n'examiner que quelques coupes centrales avant d'entraîner un modèle ?

5 Normalisation des intensités

1. **CT.** Tracer l'histogramme des intensités d'un volume CT (échelle logarithmique en ordonnée). Identifier les pics correspondants à l'air, aux poumons éventuels, aux tissus mous et à l'os.
2. Appliquer un fenêtrage $[a, b]$ puis une normalisation affine vers $[0, 1]$:

$$x \mapsto \frac{\mathrm{clip}(x, a, b) - a}{b - a}.$$

   Choisir a et b en fonction de la tâche (rate, foie, ...).
3. **IRM (si disponible).** Standardiser par volume : $x \mapsto (x - \mu_M)/\sigma_M$, où μ_M, σ_M sont la moyenne et l'écart-type calculés *sur un masque* (cerveau, corps) et non sur tout le volume.
4. Comparer les histogrammes de plusieurs volumes d'un même jeu, puis de deux jeux de données différents, avant et après normalisation. Calculer une distance entre distributions d'intensités (par exemple la distance de Wasserstein 1D avec `scipy.stats.wasserstein_distance`).

⋆ Pourquoi la normalisation z-score sur tout le volume est-elle problématique pour une IRM ? Et pourquoi les paramètres de normalisation doivent-ils être estimés sur l'ensemble d'apprentissage uniquement lorsqu'ils sont *globaux* ?

6 Rééchantillonnage

1. Calculer l'espacement médian d'un jeu de volumes. Choisir un espacement cible commun (par exemple $1.5 \times 1.5 \times 1.5$ mm).
2. **[À faire]** Rééchantillonner un volume et son masque à l'espacement cible avec `SimpleITK` (`ResampleImageFilter`) ou `MONAI` (`Spacing`). Utiliser une interpolation *linéaire* pour l'image et *plus proche voisin* pour le masque.
3. Montrer expérimentalement ce qui se passe si l'on interpole le masque de façon linéaire : quelles valeurs apparaissent ? Combien de voxels n'ont plus une valeur entière ?
4. Comparer, avant et après rééchantillonnage, la forme du volume, son espacement et le volume de l'organe en millilitres. Expliquer les différences éventuelles.

⋆ Quels sont les avantages et inconvénients de rééchantillonner à un espacement commun plus *fin* ou plus *grossier* que l'espacement médian du jeu (mémoire, perte d'information, interpolation) ?

```python
def resample(img_sitk, new_spacing, is_label=False):
    interp = sitk.sitkNearestNeighbor if is_label else sitk.sitkLinear
    # TODO: compute the new size from the old size and spacings,
    # then use sitk.Resample with the same origin and direction
    pass
```

7 Séparation train/test sans fuite d'information

1. Construire un tableau (`pandas.DataFrame`) listant, pour chaque volume : identifiant du patient, nombre de coupes, espacement, volume de l'organe.
2. **[À faire]** Mettre en place une séparation train/validation/test (par exemple 60/20/20) *par patient*, reproductible (graine fixée). Vérifier explicitement qu'aucun patient n'apparaît dans deux ensembles.
3. Stratifier la séparation (par exemple selon le volume de l'organe ou la présence de lésion) pour que les trois ensembles aient des distributions comparables. Vérifier avec un test ou un graphique.
4. Expliquer, dans une cellule de texte, l'ordre correct des étapes : séparation → estimation des paramètres de normalisation → entraînement. Pourquoi cet ordre ?

Démonstration d'une fuite de données

L'objectif est de *mesurer* l'effet d'une séparation par coupe. Construire un problème de classification binaire au niveau de la coupe : ń la coupe contient-elle l'organe segmenté ? ż.

1. Extraire toutes les coupes axiales de tous les volumes, les réduire à 32×32 pixels, les aplatir en vecteurs. Étiqueter chaque coupe selon la présence de l'organe dans le masque.
2. Entraîner un classifieur simple (régression logistique ou k plus proches voisins, `scikit-learn`) avec deux protocoles : (a) séparation *aléatoire par coupe*, (b) séparation *par patient*.
3. Comparer la précision (ou l'AUC) sur le jeu de test dans les deux cas, avec plusieurs graines pour obtenir une dispersion.

⋆ Interpréter l'écart observé. Pourquoi les coupes voisines d'un même patient ne sont-elles pas indépendantes ? Quel protocole fournit une estimation honnête de la performance sur de nouveaux patients ?

8 Pour aller plus loin (facultatif)

— Construire un `Dataset` PyTorch/MONAI qui charge, normalise, rééchantillonne et découpe en patchs $96 \times 96 \times 96$ avec un échantillonnage préférentiel des patchs contenant l'organe (par exemple un patch sur trois centré sur le premier plan).

— Évaluer le temps de chargement et l'occupation mémoire. Tester la mise en cache (`CacheDataset`).

— Visualiser quelques patchs avec leurs masques pour vérifier visuellement la cohérence géométrique.

— Tester l'effet de plusieurs augmentations (rotations, élastique, bruit, variation de contraste) sur une coupe, et décider lesquelles sont plausibles du point de vue médical.

Récapitulatif attendu dans le compte rendu

Partie	Livrable principal
DICOM	conversion en HU, trois fenêtrages, champs à anonymiser
NIfTI	géométrie, orientation, volume en mL
Visualisation	coupes avec bon rapport d'aspect, superposition
Normalisation	histogrammes avant/après, distance entre jeux
Rééchantillonnage	espacement commun, effet de l'interpolation du masque
Séparation	découpage par patient, démonstration d'une fuite